



# Universidade Tuiuti do Paraná

Credenciada por Decreto Presidencial de 07 de julho de 1997  
D.O.U. nº 128, de 08 de julho de 1997, Seção 1, página 14295

---

## Bacharelado em Ciência da Computação Projeto de Graduação

---

### Proposta de Tema

(Apêndice CC1)

#### 1. Título

Ferramenta Baseada em Linguagem de Domínio Específico para o Apoio ao Ensino de Autômatos Finitos com Simulação Visual Interativa.

#### 2. Tema

O tema deste trabalho situa-se na interseção entre compiladores e o ensino de Teoria da Computação, mais especificamente no subdomínio de autômatos finitos. O assunto central é o projeto e a implementação de uma Linguagem de Domínio Específico (DSL) textual voltada à descrição formal de Autômatos Finitos Determinísticos (AFD) e Não-Determinísticos (AFN), integrada a uma ferramenta com interface gráfica capaz de simular visualmente e de forma interativa o processamento de cadeias de entrada sobre os autômatos descritos.

O domínio de problema abordado é a dificuldade enfrentada por estudantes de Ciência da Computação na compreensão prática do funcionamento de autômatos finitos. Embora esses conceitos sejam fundamentais em disciplinas como Linguagens Formais, Teoria da Computação e Compiladores, o aprendizado costuma ocorrer de maneira predominantemente teórica e abstrata, com simulações feitas manualmente no papel. As ferramentas existentes, como o JFLAP, adotam uma abordagem exclusivamente visual baseada em arrastar e soltar, sem oferecer uma representação textual formal que possa ser versionada, reproduzida e validada sintaticamente e semanticamente.

A solução computacional proposta consiste no desenvolvimento de uma DSL externa com sintaxe própria, acompanhada de um compilador que realiza análise léxica, sintática e semântica do código escrito pelo usuário, e de uma interface gráfica interativa que renderiza o autômato descrito e anima o processo de simulação passo a passo, destacando visualmente os estados e transições percorridos durante o reconhecimento de uma cadeia.

O escopo do trabalho é delimitado ao suporte a Autômatos Finitos Determinísticos (AFD) e Não-Determinísticos (AFN), não abrangendo autômatos com pilha, máquinas de Turing, conversão automática entre modelos ou minimização de autômatos, os quais ficam como possibilidades para trabalhos futuros.

#### 3. Equipe

Aluno: Klaus Siegfried Beckmann

E-mail: klaus.beckmann@utp.edu.br

Telefone: (41) 99696-5462

---

Aluno: João Thomaz Vieira  
E-mail: joao.vieira3@utp.edu.br Telephone: (41) 99738-6125

---

#### 4. Orientador e Coorientador

Nome: Diógenes Cogo Furlan  
E-mail: diogenes.furlan@utp.br Telephone: (41) 9 9626-2121  
Nome: \_\_\_\_\_  
E-mail: \_\_\_\_\_ Telephone: \_\_\_\_\_

#### 5. Reunião de Orientação

Dia da semana: Quinta-feira Hora: 21:00  
Local: Sala de aula

---

#### 6. Resumo

##### 6.1. Introdução e Justificativa

As ferramentas existentes que se propõem a auxiliar nesse processo, como o JFLAP, oferecem interfaces gráficas baseadas em manipulação visual, onde o usuário constrói o autômato arrastando e soltando estados e conectando transições com o mouse. Embora essa abordagem seja útil, ela apresenta limitações relevantes. A descrição do autômato não é textual, o que dificulta o versionamento, a reprodução e o compartilhamento dos modelos criados. Além disso, não existe uma validação formal da estrutura do autômato com mensagens de erro claras e localizadas, como ocorre em linguagens de programação. O estudante que comete um erro na construção não recebe um diagnóstico preciso do problema, mas apenas percebe um comportamento inesperado na simulação.

Diante desse cenário, este trabalho propõe o desenvolvimento de uma ferramenta que adota uma abordagem diferente: uma Linguagem de Domínio Específico (DSL) textual para descrição de autômatos, processada por um compilador próprio, integrada a uma interface gráfica que anima a simulação passo a passo. Essa combinação permite que o estudante descreva autômatos de forma declarativa e precisa, receba diagnósticos de erro detalhados e localizados quando a descrição contiver inconsistências, e visualize o processamento de cadeias de forma animada e interativa. Um aspecto adicional de relevância é que a própria ferramenta, por ser construída com técnicas reais de compilação (análise léxica, sintática, semântica e construção de Árvore Sintática Abstrata), funciona como um exemplo prático dos conceitos que o estudante está aprendendo, conectando a teoria de autômatos à prática de compiladores em um único instrumento.

Dessa forma, a pesquisa se justifica pela ausência de uma ferramenta que integre descrição textual formal, validação com diagnósticos claros e simulação visual interativa no ensino de autômatos finitos, oferecendo uma abordagem que as soluções atuais não contemplam de maneira unificada.

##### 6.2. Problema Proposto

Como projetar uma Linguagem de Domínio Específico (DSL) textual que seja ao mesmo tempo formalmente rigorosa e intuitiva o suficiente para que estudantes de Ciência da Computação descrevam Autômatos Finitos Determinísticos (AFD) e Não-Determinísticos (AFN), e como implementar um compilador capaz de validar automaticamente a consistência dessas descrições, fornecendo mensagens de erro claras e localizadas que auxiliem o aprendizado?

De que forma a simulação visual e interativa do processamento de cadeias de entrada, com animação passo a passo dos estados e transições percorridos, pode contribuir para a compreensão prática do funcionamento de autômatos finitos, complementando a abordagem textual e formal da DSL?

Como avaliar se o protótipo desenvolvido atende aos requisitos de corretude na análise e simulação de autômatos, e se as mensagens de diagnóstico produzidas pelo compilador são de fato compreensíveis e úteis para o estudante identificar e corrigir erros na descrição?

### 6.3. Objetivos

- Desenvolver uma Linguagem de Domínio Específico (DSL) textual com sintaxe própria, projetada para a descrição declarativa de Autômatos Finitos Determinísticos (AFD) e Não-Determinísticos (AFN), contemplando a definição de estados, alfabeto, estado inicial, estados finais e função de transição, com a gramática formalizada em notação BNF/EBNF.
- Implementar um compilador para a DSL proposta que realize as fases de análise léxica, análise sintática e análise semântica, construindo uma Árvore Sintática Abstrata (AST) como representação intermediária e fornecendo mensagens de erro detalhadas com indicação da localização e da natureza do problema encontrado no código do usuário.
- Desenvolver um simulador capaz de processar cadeias de entrada sobre os autômatos descritos na DSL, registrando o caminho de estados e transições percorridos a cada passo e determinando se a cadeia é aceita ou rejeitada pelo autômato.
- Construir uma interface gráfica interativa que renderize visualmente o autômato descrito no código da DSL e que anime o processo de simulação passo a passo, destacando graficamente o estado atual, a transição sendo percorrida e o símbolo sendo consumido da cadeia de entrada, permitindo ao usuário controlar a velocidade e navegar entre os passos da simulação.
- Validar o protótipo por meio de testes com autômatos clássicos da literatura, verificando a corretude da análise e da simulação em diferentes cenários, incluindo autômatos com erros propositalmente para avaliar a qualidade e a clareza das mensagens de diagnóstico.

### 6.4. Trabalhos Relacionados

#### 6.4.1. Writing a Lexer in Rust:

Karekar (2021) descreve a implementação de um analisador léxico (lexer) em Rust, adaptando os conceitos apresentados no livro *Writing an Interpreter in Go* para a linguagem Rust. O problema abordado é a necessidade de aprender, na prática, como funciona a primeira etapa de um compilador ou interpretador: a análise léxica, responsável por transformar o código-fonte em uma sequência de tokens significativos para as fases subsequentes do processo de compilação.

A solução consiste na implementação de uma struct `Lexer` em Rust que mantém estado de leitura (posição atual, posição de lookahead e caractere corrente) e de um método `next_token()` que utiliza pattern matching para identificar o tipo de cada token. O trabalho demonstra como os tipos algébricos (enums) e o sistema de pattern matching de Rust são especialmente adequados para a

definição e o reconhecimento de tokens, substituindo com vantagem as structs e verificações em cadeia típicas de linguagens como Go ou C.

A relação com a presente proposta é de caráter técnico e de base de implementação: o compilador da DSL proposta inclui uma fase de análise léxica que pode se beneficiar diretamente das técnicas e estruturas demonstradas neste trabalho. O uso de Rust como linguagem de implementação, com seu sistema de tipos e pattern matching, é considerado uma alternativa viável para a construção das fases de análise léxica e sintática do compilador da ferramenta proposta.

#### **6.4.2. JFLAP:**

O JFLAP é um software educacional interativo escrito em Java para experimentação com tópicos de linguagens formais e teoria dos autômatos, voltado principalmente para o nível de graduação. A ferramenta permite a construção e simulação de autômatos finitos (DFA e NFA), autômatos com pilha, máquinas de Turing, além de conversões entre modelos (como NFA para DFA e minimização). O JFLAP pode ser utilizado tanto em sala de aula quanto fora dela, sendo usado durante aulas para introduzir tópicos, trabalhar exemplos e ilustrar a construção e execução de máquinas. A abordagem é exclusivamente gráfica, com o usuário construindo autômatos por meio de arrastar e soltar, sem oferecer uma representação textual formal ou linguagem de descrição própria. Também não fornece mensagens de erro localizadas com indicação de linha e coluna, como ocorre em compiladores.

#### **6.4.3. GAM:**

A GAM é uma ferramenta que fornece condições de simular diferentes tipos de máquinas abstratas a fim de executar diversas operações e conversões que são comuns nessa área de estudo. Desenvolvida na Universidade Federal de Goiás e na Universidade Federal de Lavras, a ferramenta permite a construção de AFDs, AFNs e conversões entre modelos. A GAM foi desenvolvida em C++ com interface GTK, disponível para GNU/Linux e Windows. Embora ofereça simulação e conversão de autômatos, não utiliza uma linguagem de descrição textual com compilador próprio nem fornece diagnósticos de erro baseados em análise léxica, sintática e semântica.

#### **6.4.4. Simulador de Autômatos e Máquinas de Turing (TCC UFSC):**

Este TCC aborda a escassez de sistemas de qualidade para simular o reconhecimento de sentenças por mecanismos reconhecedores, identificando que as soluções existentes frequentemente apresentam dificuldades de uso, incompletude, problemas de funcionamento e baixa extensibilidade. O autor propõe o desenvolvimento de uma aplicação web focada em quatro propriedades: facilidade de uso, completude (evitando restrições excessivas no alfabeto), corretude no reconhecimento e extensibilidade para adição de novos mecanismos. O sistema suporta autômatos finitos (DFA e NFA), autômatos de pilha (DPDA e NPDA) e autômatos linearmente limitados (LBA), oferecendo edição via diagrama de estados e via tabela de transições, reconhecimento passo a passo, rápido e múltiplo, suporte multi-idioma e persistência de autômatos. O trabalho inclui uma análise comparativa detalhada de oito ferramentas existentes (JFLAP, jFAST, Automaton Simulator, entre outras). Apesar de abrangente em tipos de autômatos e funcionalidades, o sistema não utiliza uma linguagem de descrição textual com sintaxe própria nem um compilador com diagnósticos de erro localizados, e a simulação visual não inclui animação com destaque gráfico dos estados e transições percorridos em tempo real.

#### **6.4.5. AutomatoGraph:**

O trabalho apresenta o desenvolvimento da ferramenta AutomatoGraph, voltada à representação e simulação de Autômatos Finitos Determinísticos (AFDs) como apoio ao ensino da

disciplina de Linguagens Formais e Autômatos. A motivação parte da dificuldade dos alunos em abstrair os conceitos teóricos para compreender o funcionamento dos formalismos. Desenvolvido em Java, o software permite ao usuário informar o número de estados, especificar estado inicial e estados finais, definir o alfabeto e as transições, e então testar o reconhecimento de palavras. Durante o processamento, a ferramenta exibe a sequência de transições percorridas pelo autômato. Além da simulação do AFD, o AutomatoGraph gera a Gramática Regular correspondente ao autômato inserido, diferenciando-se de outras ferramentas da época. A ferramenta foi testada por alunos da disciplina, que contribuíram com sugestões de melhoria na interface. Contudo, o sistema é limitado a AFDs (sem suporte a AFNs), não possui uma linguagem de descrição textual com compilador, não oferece animação visual com destaque gráfico dos estados durante a simulação e a entrada do autômato se dá por formulários, não por uma DSL formal.

| Trabalhos               | DSL textual | Compilador com diagnóstico | Simulação visual  | Simulação passo a passo       | Plataforma                      |
|-------------------------|-------------|----------------------------|-------------------|-------------------------------|---------------------------------|
| Writing a Lexer in Rust | Não         | Não                        | Não               | Linha de comando              | Implementação de léxico em Rust |
| JFLAP                   | Não         | Não                        | Sim               | Sim                           | Desktop (Java)                  |
| GAM                     | Não         | Não                        | Sim               | Parcial                       | Desktop (C++/GTK)               |
| Simulador UFSC          | Não         | Não                        | Sim               | Sim                           | Web                             |
| AutomatoGraph           | Não         | Não                        | Parcial (textual) | Parcial (lista de transições) | Desktop (Java)                  |

## 6.5. Produtos a serem Gerados

### 6.5.1 Artefatos de Especificação de Projeto:

P1 — Especificação formal da gramática da DSL. Será produzida a definição completa da gramática livre de contexto da linguagem de domínio específico proposta, expressa em notação BNF ou EBNF. A gramática cobrirá todas as construções da DSL, incluindo a declaração do alfabeto, dos estados, do estado inicial, dos estados finais e das funções de transição para AFDs e AFNs. Este artefato servirá como contrato formal entre a especificação da linguagem e sua implementação no compilador.

P2 — Diagramas de arquitetura da ferramenta. Serão elaborados diagramas estruturais e de fluxo que representam a organização interna do sistema. Entre os diagramas previstos estão: (a) o pipeline de compilação, ilustrando as fases de análise léxica, sintática e semântica e a geração do modelo interno do autômato; (b) o diagrama de componentes da interface gráfica, separando o módulo de renderização do grafo e o módulo de controle da simulação; e (c) o fluxograma do algoritmo de simulação passo a passo para AFDs e AFNs, incluindo o tratamento do não-determinismo via subconjunto de estados ativos.

### 6.5.2 Artefatos de Implementação e Software:

P3 — Protótipo funcional da ferramenta. O principal produto da pesquisa é a implementação completa da ferramenta descrita na proposta. O protótipo reunirá: (a) um editor de código com realce de sintaxe para a DSL desenvolvida; (b) um compilador que realiza análise léxica, sintática e semântica do programa fonte, emitindo mensagens de erro e aviso localizadas com indicação precisa

de linha e coluna; e (c) uma interface gráfica interativa que renderiza o autômato descrito na forma de um grafo dirigido e anima o processamento de cadeias de entrada passo a passo, destacando visualmente os estados e transições percorridos. O suporte abrangerá AFDs e AFNs.

P4 — Código-fonte documentado e repositório público. O código-fonte completo da ferramenta será disponibilizado em repositório público (GitHub), organizado em módulos com documentação inline. O repositório incluirá instruções de instalação e build, descrição da estrutura do projeto e histórico de versões, permitindo que a ferramenta seja reproduzida, auditada e estendida por outros pesquisadores ou educadores.

### **6.5.3 Artefatos de Validação e Documentação:**

P5 — Conjunto de casos de teste automatizados. Será construída uma suíte de testes automatizados cobrindo as três fases do compilador e o módulo de simulação. Os casos de teste incluirão: programas com erros léxicos, sintáticos e semânticos deliberados, com verificação das mensagens de diagnóstico esperadas; autômatos válidos de AFD e AFN com verificação do modelo interno gerado; e pares (autômato, cadeia de entrada) com verificação do resultado de aceitação ou rejeição. Os resultados do simulador serão comparados com saídas equivalentes geradas pelo JFLAP, utilizado como referência de correção.

P6 — Relatório de validação técnica. Documento que consolida os resultados da execução da suíte de testes, incluindo índice de cobertura de código, lista de casos de borda identificados e tratados, e análise comparativa entre os resultados do simulador proposto e os do JFLAP. O relatório evidenciará a correção funcional da ferramenta antes de sua aplicação com usuários.

P7 — Estudo de caso com estudantes. Será conduzida uma aplicação controlada da ferramenta com estudantes de graduação matriculados na disciplina de Linguagens Formais ou Teoria da Computação. O estudo de caso coletará dados de uso observacional, erros cometidos durante a interação, tempo de execução de tarefas representativas e percepção de utilidade por meio de questionário estruturado. Os resultados serão analisados e reportados com o objetivo de identificar pontos de melhoria e fornecer evidências preliminares de eficácia pedagógica da ferramenta.

P8 — Manual do usuário. Será produzido um manual dirigido ao estudante, descrevendo de forma clara e progressiva: a sintaxe completa da DSL com exemplos comentados de AFDs e AFNs; a interpretação das mensagens de diagnóstico emitidas pelo compilador; e o uso da interface de simulação, incluindo os controles de execução passo a passo, a leitura do destaque visual de estados e transições, e a entrada de múltiplas cadeias de teste. O manual será distribuído junto ao repositório da ferramenta

## 6.6. Cronograma:

O projeto de graduação será desenvolvido ao longo de 12 (doze) meses, distribuídos em dois semestres letivos. O primeiro semestre (PG1), compreendendo o período de fevereiro a julho de 2026, concentra-se nas etapas de levantamento teórico, especificação formal da linguagem e implementação das fases iniciais do compilador. O segundo semestre (PG2), de setembro de 2026 a dezembro de 2026, abrange a conclusão da implementação, a condução do estudo de caso, a elaboração da documentação final e a defesa do trabalho.

| Fases do Projeto                                         | Fevereiro | Março     | Abril     | Maiο       | Junho | Julho |
|----------------------------------------------------------|-----------|-----------|-----------|------------|-------|-------|
| Elaboração e entrega da proposta do PG I                 |           | 21/03/26  |           |            |       |       |
| Retorno da proposta do PG I                              |           | 5ª semana |           |            |       |       |
| Reentrega da proposta do PG I (se NOK)                   |           |           | 06/04/26  |            |       |       |
| Retorno da reentrega da proposta (se NOK)                |           |           | 7ª semana |            |       |       |
| Fundamentação Teórica                                    |           | X         | X         | X          |       |       |
| Trabalhos Relacionados                                   |           | X         | X         | X          |       |       |
| Metodologia de Construção da Solução (versão preliminar) |           |           | X         | X          | X     |       |
| Ferramentas para Implementação                           |           |           |           | X          | X     |       |
| Entrega do PG I — checkpoint                             |           |           |           | 09/05/26   |       |       |
| Retorno do checkpoint<br>sem nota)                       |           |           |           | 12ª semana |       |       |

|                                                              |  |  |  |  |          |                      |
|--------------------------------------------------------------|--|--|--|--|----------|----------------------|
| Entrega para avaliação do PG I                               |  |  |  |  | 13/06/26 |                      |
| Retorno da Avaliação do PG I<br>(3 avaliadores, com nota)    |  |  |  |  | 22-27/06 |                      |
| Entrega para reavaliação do PG I<br>(somente nota 4,0 – 6,9) |  |  |  |  |          | 7 dias após<br>aval. |

## 6.7. Referências

JUKEMURA, Anibal S.; NASCIMENTO, Hugo A. D.; UCHÔA, Joaquim Quinteiro. **GAM: um simulador para auxiliar o ensino de linguagens formais e de autômatos**. 2005. Disponível em: [https://www.researchgate.net/publication/228879910\\_GAM-Um\\_Simulador\\_para\\_Auxiliar\\_o\\_Ensino\\_de\\_Linguagens\\_Formais\\_e\\_de\\_Automatos](https://www.researchgate.net/publication/228879910_GAM-Um_Simulador_para_Auxiliar_o_Ensino_de_Linguagens_Formais_e_de_Automatos). Acesso em: 21 mar. 2026.

KAREKAR, Mohit. **Writing a lexer in Rust**. 2021. Disponível em: <https://mohitkarekar.com/posts/pl/lexer/>. Acesso em: 21 mar. 2026.

KIENETZ, Eduardo Bacchi; CANAL, Ana Paula. **Simulador de autômatos finitos determinísticos – Automatograph**. 2004. Disponível em: <https://periodicos.ufn.edu.br/index.php/disciplinarumNT/article/view/1176/1113>. Acesso em: 21 mar. 2026.

NUNES, Ghabriel Calsa. **Simulador de autômatos e máquinas de Turing**. Disponível em: <https://repositorio.ufsc.br/handle/123456789/182184>. 2017. Acesso em: 21 mar. 2026.

RODGER, Susan H.; FINLEY, Thomas W. **JFLAP**. Disponível em: <https://www.jflap.org/>. Acesso em: 21 mar. 2026.

Curitiba, 21 de Março de 2026.